Sistem Informasi Seluler

Materi Week 2

Interaksi, input, dan daftar data di Flutter



Poin Pembelajaran Tambahan

- StatelessWidget vs StatefulWidget
- setState() dan alur pembaruan tampilan
- TextField, controller, dan Form
- Image, Card, dan ListView
- Tantangan: aplikasi daftar nama

StatelessWidget & StatefulWidget

Sejauh ini widget yang kita buat bersifat statis. Agar aplikasi bisa merespons aksi user, seperti mencentang checkbox atau menekan tombol, Flutter perlu tahu kapan tampilan harus digambar ulang. Di sinilah konsep state dan setState() berperan.

State | setState() | build() | Rebuild

StatelessWidget vs StatefulWidget

StatelessWidget

Tampilan tetap

- Tidak menyimpan data yang berubah
- Hanya satu class dengan method build()
- Contoh: MyApp, judul, dan ikon

StatefulWidget

Tampilan bisa berubah

- Menyimpan state di class State
- Memakai setState() untuk memperbarui UI
- Contoh: Checkbox, Switch, Slider, form

State adalah data yang bisa berubah selama aplikasi berjalan.

Sistem Informasi Seluler

Kenapa Checkbox Tidak Tercentang?

Tanpa setState()

```
onChanged: (bool? newValue) {
  isChecked = newValue!;
  print('Checkbox: $isChecked');
},
```

Dengan setState()

```
onChanged: (bool? newValue) {
  setState(() {
    isChecked = newValue!;
  });
},
```

Mengubah variabel saja tidak cukup. Flutter baru menggambar ulang layar ketika setState() dipanggil, jadi tanpa setState() nilai isChecked berubah tetapi kotak centang tetap kosong. Kode pada slide Checkbox, Radio, Switch, Slider, dan Dropdown perlu dibungkus setState().

Sistem Informasi Seluler

Struktur StatefulWidget

Class 1: Widget

```dart
class CounterPage extends StatefulWidget {
  const CounterPage({super.key});

  @override
  State<CounterPage> createState() ⇒
      _CounterPageState();
}
```

Class 2: State

```dart
class _CounterPageState
    extends State<CounterPage> {
  int counter = 0; // state

  @override
  Widget build(BuildContext context) {
    return Text('Counter: $counter');
  }
}
```

StatefulWidget terdiri dari dua class. Class pertama adalah widget itu sendiri dan membuat State lewat createState(). Class kedua adalah State, tempat variabel yang bisa berubah disimpan dan method build() ditulis.

setState()

```dart
Text('Jumlah: $counter'),

ElevatedButton(
  onPressed: () {
    setState(() {
      counter++;
    });
  },
  child: Text('Tambah'),
),
```

setState() memberi tahu Flutter bahwa state telah berubah, sehingga method build() dijalankan ulang dan layar diperbarui. Ubah nilai variabel di dalam setState(), bukan di luarnya.

Cara Kerja setState()

1

Aksi user

Menekan tombol atau mencentang checkbox

→

2

setState()

Nilai state diubah dan Flutter diberi tahu

→

3

build()

Dijalankan ulang oleh Flutter

→

4

UI baru

Layar menampilkan nilai terbaru

Tanpa setState(): nilai variabel berubah, tetapi tampilan tidak ikut berubah.

Selection Widget Selalu Memakai setState()

Widget	Variabel state	Di dalam setState()
Checkbox	bool isChecked	isChecked = newValue!;
Radio	int selectedValue	selectedValue = newValue!;
Switch	bool isSwitched	isSwitched = newValue;
Slider	double sliderValue	sliderValue = newValue;
Dropdown	String dropdownValue	dropdownValue = newValue!;

Pola yang sama: ubah variabel di dalam setState(() { ... }).

Input Teks

Checkbox, Radio, dan Switch hanya memungkinkan user memilih. Hampir semua aplikasi juga membutuhkan user mengetik, seperti form login, kolom pencarian, atau pengisian data. Flutter menyediakan TextField untuk kebutuhan ini.

TextField | TextEditingController | Form | TextFormField

TextField

```dart
TextField(
  decoration: InputDecoration(
    labelText: 'Nama',
    hintText: 'Masukkan nama lengkap',
    border: OutlineInputBorder(),
    prefixIcon: Icon(Icons.person),
  ),
  onChanged: (value) {
    print('Isi: $value');
  },
),
```

TextField adalah widget untuk mengetik teks. Properti decoration dengan InputDecoration mengatur label, hint, border, dan ikon. Callback onChanged dipanggil setiap kali isi teks berubah.

Sistem Informasi Seluler

TextEditingController

Membuat dan memasang

```
final namaController =
    TextEditingController();

TextField(
  controller: namaController,
),
```

Membaca teks dan dispose

```
onPressed: () {
  print(namaController.text);
},

@override
void dispose() {
  namaController.dispose();
  super.dispose();
}
```

TextEditingController menyimpan isi TextField sehingga bisa dibaca lewat .text atau diubah dari kode. Buat controller di dalam class State, lalu buang dengan dispose() agar tidak memakan memori.

Properti Penting TextField

Properti	Fungsi	Contoh
decoration	Label, hint, border, ikon	InputDecoration(labelText: 'Email')
controller	Membaca dan mengatur isi	controller: emailController
obscureText	Menyembunyikan teks (password)	obscureText: true
keyboardType	Jenis keyboard yang muncul	TextInputType.emailAddress
maxLines	Jumlah baris input	maxLines: 3
onChanged	Dipanggil saat teks berubah	onChanged: (value) {}

Form & TextFormField

```dart
final _formKey = GlobalKey<FormState>();

Form(
  key: _formKey,
  child: TextFormField(
    decoration: InputDecoration(labelText: 'Email'),
    validator: (value) {
      if (value == null || value.isEmpty) {
        return 'Email wajib diisi';
      }
      return null;
    },
  ),
),
```

Form mengelompokkan beberapa input, dan TextFormField menambahkan properti validator. Jika validator mengembalikan teks, teks itu tampil sebagai pesan error; jika mengembalikan null, input dianggap valid.

Validasi Form

```dart
ElevatedButton(
  onPressed: () {
    if (_formKey.currentState!.validate()) {
      print('Form valid');
    }
  },
  child: Text('Kirim'),
),
```

Memanggil validate() menjalankan semua validator di dalam Form. Kode di dalam if hanya berjalan bila semua input valid, misalnya untuk menyimpan data atau berpindah halaman.

Widget Tampilan Lainnya

Aplikasi nyata hampir selalu menampilkan gambar dan daftar data. Image menampilkan gambar, Card membungkus konten dalam kartu, dan ListView menyusun banyak item yang bisa di-scroll.

Image Card ListTile ListView

Sistem Informasi Seluler

Image

Image.asset

```dart
Image.asset(
  'assets/images/logo.png',
  width: 120,
  height: 120,
),
```

Image.network

```dart
Image.network(
  'https://picsum.photos/200',
  width: 120,
  height: 120,
  fit: BoxFit.cover,
),
```

Image.asset memakai file yang ada di proyek dan harus didaftarkan di pubspec.yaml pada bagian flutter: assets:. Image.network memuat gambar dari internet lewat URL.

Card & ListTile

```dart
Card(
  child: ListTile(
    leading: Icon(Icons.person),
    title: Text('Budi Santoso'),
    subtitle: Text('Teknik Informatika'),
    trailing: Icon(Icons.chevron_right),
  ),
),
```

Card membungkus konten dalam kartu dengan sudut membulat dan bayangan. ListTile menyediakan susunan siap pakai: leading (kiri), title, subtitle, dan trailing (kanan). Icon menampilkan ikon bawaan dari Icons.

ListView

```dart
ListView(
  children: [
    ListTile(title: Text('Item 1')),
    ListTile(title: Text('Item 2')),
    ListTile(title: Text('Item 3')),
  ],
),
```

ListView menyusun widget secara vertikal seperti Column, tetapi bisa di-scroll saat isinya melebihi layar. Cocok untuk daftar dengan jumlah item yang sedikit dan tetap.

ListView.builder

```dart
final daftarNama = ['Ani', 'Budi', 'Citra'];

ListView.builder(
  itemCount: daftarNama.length,
  itemBuilder: (context, index) {
    return Card(
      child: ListTile(
        title: Text(daftarNama[index]),
      ),
    );
  },
),
```

ListView.builder membuat item hanya saat dibutuhkan, sehingga efisien untuk data yang banyak. itemCount menentukan jumlah item, dan itemBuilder dipanggil untuk setiap index.

Tantangan: Aplikasi Daftar Nama

1. TextField dengan controller untuk mengetik nama
2. ElevatedButton Tambah yang memanggil setState()
3. ListView.builder untuk menampilkan nama dalam Card
4. Bonus: Form dan validator agar nama tidak boleh kosong

```
daftarNama.add(namaController.text);
```

Material App

```dart
class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Basic MaterialApp Example',
      theme: ThemeData(
        colorScheme: ColorScheme.fromSeed(
          seedColor: Colors.blue, // Warna dasar
        ),
      ),
      home: MyHomePage(), // Halaman utama
    );
  }
}
```

MaterialApp adalah widget utama untuk aplikasi bergaya Material Design. Untuk tema warna, gunakan colorScheme dengan ColorScheme.fromSeed(): Flutter menyusun satu set warna serasi dari satu warna dasar. Cara lama dengan primarySwatch sudah tidak disarankan.

Container

```dart
body: Container(
  color: Colors.blue, // Latar biru
  width: 100, // Lebar
  height: 100, // Tinggi
  child: Center(
    child: Text('Inside Container'), // Isi
  ),
),
```

Container adalah widget serbaguna yang dapat digunakan untuk membuat kotak pembungkus bagi widget lain. Container memungkinkan Anda menambahkan properti seperti warna latar belakang, margin, padding, border, atau ukuran tertentu ke child-nya.

Invisible Widget

Invisible Widget adalah widget yang tidak menampilkan konten sendiri; tugasnya mengatur kerangka dan tata letak. Row dan Column menyusun widget secara horizontal dan vertikal, Container membungkus widget lain dengan warna dan properti tambahan, Center menempatkan satu widget di tengah, dan Scaffold menyediakan kerangka halaman.

Catatan

Scaffold memang memunculkan AppBar, body, dan FloatingActionButton di layar, tetapi elemen-elemen itu adalah widget anak yang kita pasang. Scaffold sendiri berperan sebagai kerangka.